



西安交通大学
XI'AN JIAOTONG UNIVERSITY

浙江大学软件学院 夏令营实践项目技术报告

任务 7. 桌面视觉智能体任务

分营名称:	大模型与工业智能
姓名:	常煜杰
学院:	人工智能学院
专业:	人工智能实验班
学号:	2204112910

2025 年 7 月 4 日

摘要

本实验聚焦于借助视觉语言模型（VLM）实现桌面自动化任务，采用 Qwen2.5-VL 模型分三阶段展开研究。

在阶段一，先完成 Qwen2.5-VL 预训练模型的本地环境部署与加载，接着输入指定桌面应用截图，对模型在图像描述（Captioning）和视觉问答（VQA）任务中的表现进行复现、测试与记录。实验发现模型虽信息识别准确率达 100%，但存在“注意力”涣散、细粒度图片理解困难等能力边界。

阶段二着重实现自然语言指令到 UI 元素坐标的转换。通过 pyautogui 和 pyperclip 库完成屏幕操作的 Python 编程，将操作指令整合成模型可理解的格式（如 Double Click、Left Click 等），并针对指令生成准确率低的问题（初始仅 70.83%）进行优化提示词的尝试，成功将准确率提升至 100%。在 UI 定位方面，尝试使用双模型二次筛选与过引入三模型交互优化定位逻辑，但因局部截图信息理解困难未达预期。

阶段三构建桌面自动化流程模拟智能体。先组装“操作 - 定位(Operation-Location)”智能体，其工作流为“截取屏幕 → 推理操作与区域 → 执行操作”，但存在定位不准导致死循环问题；后优化为“动作识别 - 行动 - 预期 (Thought-Action-Expectation)”智能体，通过三模型循环交互（判断操作预期 → 生成动作与区域 → 描述预期）提升鲁棒性。测试显示，TAE 智能体在 MISSION1 执行中准确率达 60%，较 OL 智能体（20%）显著提升，且能在操作出错时提前终止循环，但对小目标元素（如代码运行按钮）定位仍存在不足。

本实验的所有实践部分均可复现且有数据记录，涉及图片、文档、代码、数据已上传 github 仓库，网址 https://github.com/AlenBarry/ZJU_7

目 录

1 实验内容	4
2 实验原理	5
2.1 Qwen2.5-VL 模型	5
2.2 GUI 任务与多模态大模型	7
3 实验环境	8
4 环境配置与模型部署	9
4.1 环境配置	9
4.2 gradio 部署	10
4.3 参数部署	12
4.4 API 部署	12
5 阶段一：图像描述与视觉问答任务	14
5.1 数据集选择与制作	14
5.2 图像描述 Captioning	15
5.2.1 信息识别的准确性	15
5.2.2 能力边界：“注意力”涣散	15
5.2.3 能力边界：相似图片区分	17
5.2.4 可能的解决措施	18
5.3 视觉问答 (Visual Question Answering)	18
5.3.1 顶层应用解析	18
5.3.2 能力边界：细粒度图片理解	19
5.3.3 能力边界：上下文关联	21
5.3.4 启示	21
6 阶段二：自然语言指令到 UI 元素坐标的转换	21
6.1 屏幕操作	21
6.1.1 操作指令的 python 实现	21
6.1.2 操作指令的可理解格式	22
6.2 屏幕的 UI 定位	24
6.2.1 电脑屏幕 UI	24
6.2.2 屏幕 UI 的模型定位	24

7 阶段三：桌面自动化流程模拟	26
7.1 操作-定位 (Operation-Location)GUI 智能体	26
7.2 动作识别-行动-预期 (Thought-Action-Expectation)GUI 智能体	28
7.3 智能体任务测试与对比	28
8 总结	31

1 实验内容

本考核项目旨在探索如何利用轻量级视觉语言模型（Vision-Language Model, VLM）完成桌面自动化任务。具体而言，本考核将使用 Qwen2.5-VL-3B 模型，通过一系列任务，引导学生从理解屏幕内容、定位 UI 元素，逐步构建一个能执行简单自动化流程的视觉智能体。本考核共分三个考核阶段，后一个阶段需要在前一个阶段完成的基础上进行。学生可以根据自身水平和进度选择完成到哪一阶段。

- **阶段一：**查找并下载 Qwen2.5-VL-3B 预训练模型（或其衍生版本）。学生需在本地完成环境部署与模型加载，并调用模型复现其基础能力。具体任务是，输入指定的桌面应用截图，测试并记录模型在图像描述（Captioning）和视觉问答（VQA）两个任务上的表现，并对结果进行分析。
- **阶段二：**在上一阶段基础上，利用模型的视觉定位（Grounding）能力，实现从自然语言指令到 UI 元素坐标的转换。要求学生编写代码，将“点击确定按钮”这类指令转化为模型可理解的格式，调用模型后，再从其输出中解析出目标元素的边界框坐标。最后需将坐标可视化地绘制在原图上，以验证定位效果。
- **阶段三：**本阶段为开放性探索任务。要求学生综合前两阶段的能力，设计并实现一个桌面简单任务自动化流程的模拟。具体方式为：设定一个多步操作任务（如用户登录），通过“观察截图、调用模型获取下一步操作的坐标、输出操作指令（如 TYPE 或 CLICK）”的循环，生成完成整个任务的指令序列，以此验证智能体的工作流。
- **要求：**交付物为实验报告文档。文档撰写要求图文结合，给出部署环境、部署过程、prompt 设计，详述部署过程中遇到的问题以及解决方法，分析所选用模型在所给定任务上的挑战和能力边界（能与不能），所列出的发现和结论需要有实验结果支撑，给出进一步解决挑战、提升能力边界的思考另外考虑到部署硬件异构，模型选用不局限于 Qwen，大家可自行寻找合适部署的多模态大模型智能体进行验证，注重在实验能力、实验结果分析能力、挑战分析、陈述的逻辑条理性、缜密程度等。

2 实验原理

要深入地完成任务，就要深入理解工具和问题，即 Qwen2.5-VL 大模型与 GUI 任务。作者重点调研了 *Qwen2.5-VL Technical Report* 和 *ScreenSpot-Pro: GUI Grounding for Professional High-Resolution Computer Use* 两篇文章，结果如下

2.1 Qwen2.5-VL 模型

Qwen2.5-VL 是阿里巴巴通义千问系列推出的最新视觉语言模型，代表了多模态大模型领域的重大技术突破。该模型在基础能力与创新功能方面均有显著提升，具备强大的图像、视频和文档理解能力，支持对象定位、文档解析以及小时级长视频的秒级事件定位。其“原生动态分辨率”机制和“动态帧率采样”技术优化了空间与时间维度的信息处理，提升了模型的适应性与准确性。架构上，Qwen2.5-VL 由基于 Qwen2.5 的大型语言模型、改进的 Vision Transformer 视觉编码器以及高效的视觉-语言融合模块组成，结合窗口注意力机制与多模态旋转位置嵌入（MRoPE），实现了更精准的跨模态推理。训练方面，采用监督微调（SFT）与直接偏好优化（DPO）相结合的方式，并通过拒绝采样不断迭代优化数据质量。在多个基准测试中，Qwen2.5-VL 表现优异，如在 MMbench-EN 上得分 88.6，在 MathVista 上获得 74.8 分，OCR 性能超越 Gemini 1.5-Pro，在 ScreenSpot Pro 上达到了 43.6% 的准确率，远超其他模型，充分体现了其在真实世界复杂任务中的强大表现力与应用潜力。

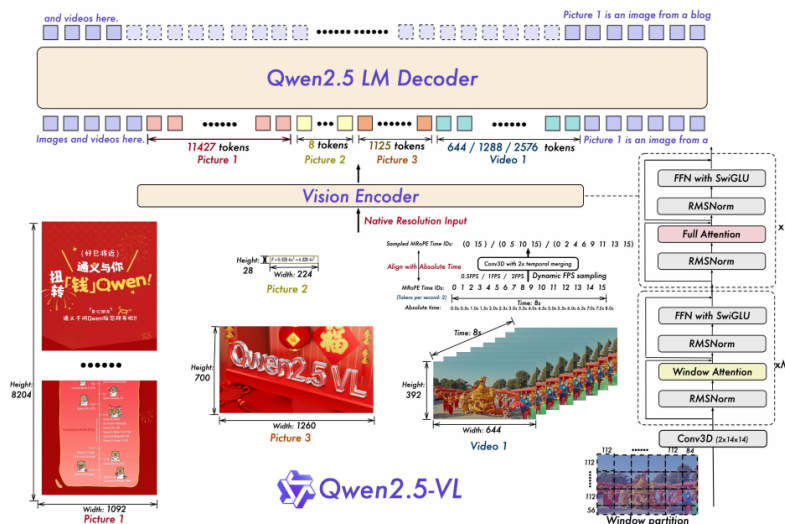


图 1: Qwen2.5-VL 模型架构

要实现阶段三的功能，有两个核心要求至关重要：模型能够理解图像的含义，以及模型能够理解指令的含义。在官网 <https://qwenlm.github.io/zh/blog/qwen2.5-vl> 上，可以看到这两个核心功能的 Demo，如图 2、图 3 所示：

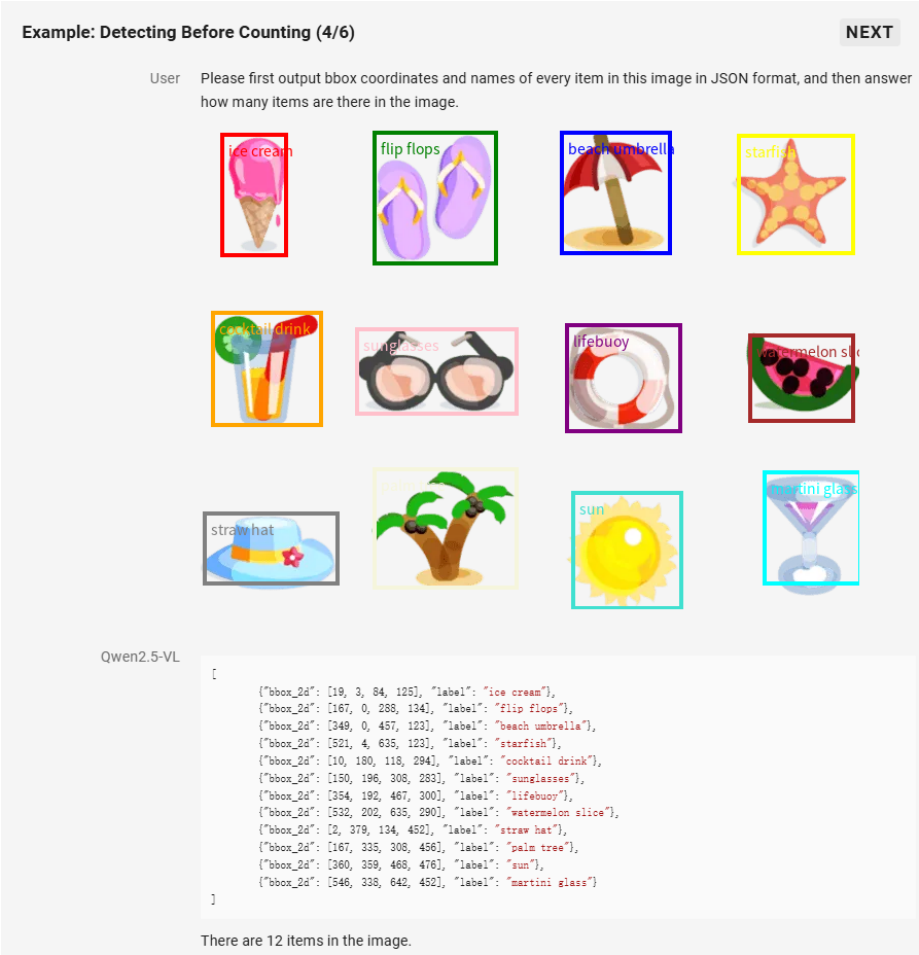


图 2: 目标检测 Demo

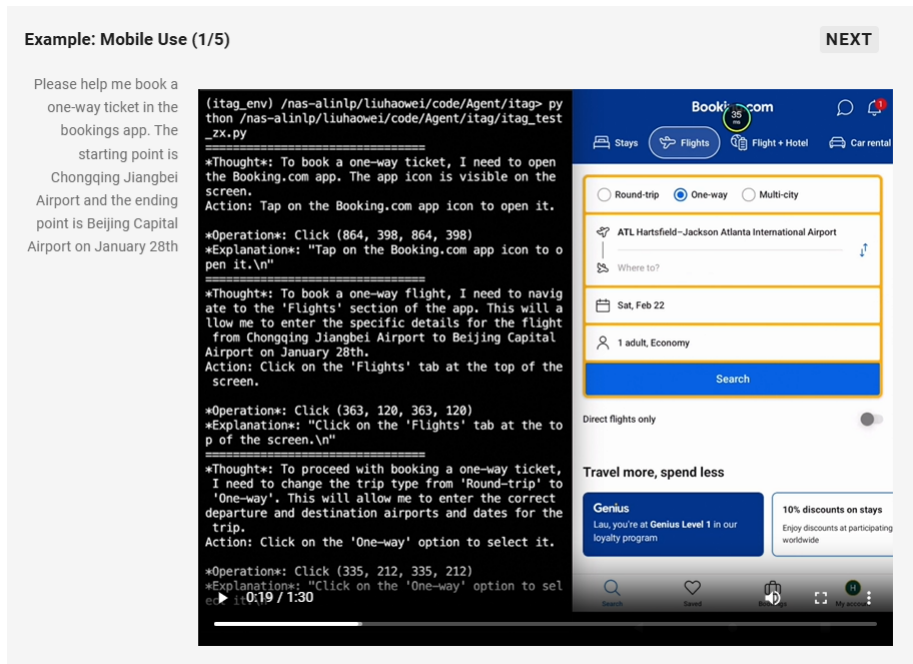


图 3: GUI 智能体 Demo

2.2 GUI 任务与多模态大模型

GUI（图形用户界面）任务是指通过理解和操作计算机应用程序的可视化界面来完成特定功能的任务，例如点击按钮、输入文本、选择菜单等。这些任务通常需要模型具备将自然语言指令与界面上的视觉元素进行精准对齐的能力。当前，随着多模态大语言模型（MLLMs）的发展，GUI 任务在通用场景（如网页浏览和移动设备使用）中取得了显著进展。然而，在专业领域的高分辨率复杂环境中，现有模型的表现仍然较差，即使是专门设计的模型，其准确率也远未达到实际应用的理想水平。此外，图标定位、小目标识别以及专业工作流程中的复杂交互仍然是主要挑战。

2024 年，新加坡国立大学的 Kaixin Li 等人提出了 Screenshot Pro，一个用于评估专业高分辨率计算机环境中图形用户界面（GUI）定位能力的新基准。与以往的基准不同，ScreenSpot-Pro 包含了来自 23 种应用程序的真实高分辨率图像和任务，这些应用程序涵盖了 5 种行业类型以及常见的 3 种操作系统使用场景。作者提出了一种基于搜索的方法 ScreenSeeker，该方法通过逐步缩小搜索空间来提高准确性，从而在没有额外训练的情况下实现了显著的性能提升。论文还讨论了 GUI 定位中的关键挑战，包括专业应用所需的高分辨率、较小的目标尺寸以及专业用户的复杂工作流程。

论文指出，当前的大模型在 ScreenSpot-Pro 上的表现较差：

Model	Development					Creative						CAD				Scientific				Office			OS		Avg		
	AS	PyC	VSC	VM	UE	PS	BI	PR	DR	AI	FL	CADSW	Inv	Qrs	Vvd	MAT	Org	Evw	Stt	PPT	Exc	Wrd	Lux	mac		Win	
OS-Atlas-7B	8.8	15.4	25.5	34.1	22.9	17.6	22.5	17.3	27.3	3.2	10.5	2.9	3.9	2.9	13.3	26.3	23.7	11.3	54.0	12.2	22.0	12.5	44.0	20.0	20.0	12.3	18.9
UGround (7B)	7.5	7.7	21.8	31.7	20.0	21.6	25.4	17.3	11.4	0.0	14.0	2.9	0.0	7.1	15.6	28.7	23.7	6.5	46.0	0.0	25.6	15.6	36.9	18.0	12.3	2.5	16.5
AriaUI (MOE, 3.9B active)	0.0	3.8	21.8	2.4	0.0	27.5	26.8	17.3	2.3	0.0	12.3	0.0	1.3	1.4	20.0	17.5	21.5	1.6	44.0	6.1	6.1	1.6	36.9	2.0	3.1	2.5	11.3
ShowUI (2B)	3.8	7.7	5.5	22.0	11.4	5.9	7.0	5.8	0.0	3.2	3.5	0.0	0.0	1.4	15.6	5.0	8.6	12.9	16.0	6.1	9.8	6.3	22.6	4.0	10.8	4.9	7.7
CogAgent (18B)	2.5	5.1	16.4	9.8	2.9	11.8	7.0	7.7	0.0	0.0	5.3	0.0	1.3	0.0	11.1	18.8	16.1	1.6	34.0	2.0	6.1	0.0	21.4	2.0	4.6	2.5	7.7
OS-Atlas-4B	1.3	1.3	12.7	2.4	0.0	0.0	2.8	1.9	2.3	3.2	5.3	0.0	0.0	1.4	2.2	3.8	7.5	3.2	20.0	0.0	4.9	0.0	8.3	6.0	0.0	3.7	3.7
MiniCPM-V (7B)	0.0	2.6	9.1	2.4	0.0	3.9	0.0	3.8	0.0	0.0	0.0	0.0	0.0	0.0	6.7	11.3	2.2	1.6	18.0	0.0	4.9	0.0	3.6	0.0	3.1	3.7	3.0
Qwen2-VL-7B	0.0	0.0	5.5	0.0	2.9	2.0	0.0	0.0	0.0	0.0	1.8	0.0	0.0	0.0	2.2	1.3	2.2	0.0	12.0	2.0	2.4	0.0	6.0	2.0	0.0	0.0	1.6
SeeClick (7B)	0.0	0.0	0.0	2.4	0.0	0.0	1.4	1.9	0.0	0.0	0.0	2.9	0.0	5.7	0.0	0.0	0.0	0.0	8.0	2.0	0.0	0.0	2.4	2.0	1.5	1.2	1.1
GPT-4o	0.0	1.3	0.0	2.4	2.9	2.0	0.0	0.0	0.0	0.0	0.0	0.0	1.3	2.9	0.0	1.3	2.2	0.0	2.0	0.0	0.0	1.6	1.2	0.0	0.0	0.0	0.8
QwenVL-7B	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	2.0	0.0	0.0	0.0	0.0	0.0	0.0	0.1

Table 2: Model Performance by Software. The abbreviations used in the table are defined in Table 1.

Model	Development			Creative			CAD			Scientific			Office			OS			Avg		
	Text	Icon	Avg	Text	Icon	Avg	Text	Icon	Avg	Text	Icon	Avg	Text	Icon	Avg	Text	Icon	Avg	Text	Icon	Avg
OSAtlas-7B	33.1	1.4	17.7	28.8	2.8	17.9	12.2	4.7	10.3	37.5	7.3	24.4	33.9	5.7	27.4	27.1	4.5	16.8	28.1	4.0	18.9
UGround (7B)	26.6	2.1	14.7	27.3	2.8	17.0	14.2	1.6	11.1	31.9	2.7	19.3	31.6	11.3	27.0	17.8	0.0	9.7	25.0	2.8	16.5
AriaUI (MOE, 3.9B active)	16.2	0.0	8.4	23.7	2.1	14.7	7.6	1.6	6.1	27.1	6.4	18.1	20.3	1.9	16.1	4.7	0.0	2.6	17.1	2.0	11.3
CogAgent (18B)	14.9	0.7	8.0	9.6	0.0	5.6	7.1	3.1	6.1	22.2	1.8	13.4	13.0	0.0	10.0	5.6	0.0	3.1	12.0	0.8	7.7
ShowUI (2B)	16.9	1.4	9.4	9.1	0.0	5.3	2.5	0.0	1.9	13.2	7.3	10.6	15.3	7.5	13.5	10.3	2.2	6.6	10.8	2.6	7.7
OSAtlas-4B	7.1	0.0	3.7	3.0	1.4	2.3	2.0	0.0	1.5	9.0	5.5	7.5	5.1	3.8	4.8	5.6	0.0	3.1	5.0	1.7	3.7
MiniCPM-V (7B)	7.1	0.0	3.7	2.0	0.0	1.2	4.1	1.6	3.4	8.3	0.0	4.7	2.8	3.8	3.0	3.7	1.1	2.6	4.5	0.7	3.0
Qwen2-VL-7B	2.6	0.0	1.3	1.5	0.0	0.9	0.5	0.0	0.4	6.3	0.0	3.5	3.4	1.9	3.0	0.9	0.0	0.5	2.5	0.2	1.6
SeeClick (7B)	0.6	0.0	0.3	1.0	0.0	0.6	2.5	0.0	1.9	3.5	0.0	2.0	1.1	0.0	0.9	2.8	0.0	1.5	1.8	0.0	1.1
GPT-4o	1.3	0.0	0.7	1.0	0.0	0.6	2.0	0.0	1.5	2.1	0.0	1.2	1.1	0.0	0.9	0.0	0.0	0.0	1.3	0.0	0.8
QwenVL-7B	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.7	0.0	0.4	0.0	0.0	0.0	0.0	0.0	0.0	0.1	0.0	0.1

Table 3: Performance breakdown of various models across application categories on ScreenSpot-Pro.

图 4: Screenshot Pro 各模型性能

而 Qwen2.5-VL 系列模型则是当前性能最优异的：

Table 9: Performance of Qwen2.5-VL and other models on GUI Agent benchmarks.

Benchmarks	GPT-4o	Gemini 2.0	Claude	Aguvis-72B	Qwen2-VL-72B	Qwen2.5-VL-72B
ScreenSpot	18.1	84.0	83.0	89.2	-	87.1
ScreenSpot Pro	-	-	17.1	23.6	1.6	43.6
Android Control High _{EM}	20.8	28.5	12.5	66.4	59.1	67.36
Android Control Low _{EM}	19.4	60.2	19.4	84.4	59.2	93.7
AndroidWorld _{SR}	34.5% (SoM)	26% (SoM)	27.9%	26.1%	6% (SoM)	35%
MobileMiniWob++ _{SR}	61%	42% (SoM)	61% (SoM)	66%	50% (SoM)	68%
OSWorld	5.03	4.70	14.90	10.26	2.42	8.83

图 5: Qwen2.5-VL-32B 模型在 GUI 任务性能

因此，本实验主要围绕 Qwen2.5-VL 系列模型展开。

3 实验环境

- **本地环境：** CPU 型号 Intel64 Family 6 Model 186 Stepping 2, 28 核
- **服务器环境：** GPU 型号 NVIDIA GeForce RTX 3090, 10496 核, 显存 24G, 共四张；NVIDIA 版本 515.43.04, CUDA 版本 11.7
- **模型硬件要求：** Qwen2.5-VL 模型有 3B、7B、72B 三个版本, 参数

量分别约为 30 亿、70 亿、720 亿，显存需求为 8G+、20G+、200G+（第三者几乎不可能在不并行的情况下推理）

综上，在本机部署时采取 3B 版本，在服务器上部署时采取 7B 版本。

4 环境配置与模型部署

配置与部署的必要文件可以在 <https://github.com/QwenLM/Qwen2.5-VL> 找到。作者在进行 Qwen2.5-VL 模型的本地部署过程中遇到了较多的复杂问题，在 csdn 网站上搜索到若干教程与解决方案，在此进行罗列：

- <https://blog.csdn.net/cscodeer/article/details/146479859>
- <https://blog.csdn.net/u014288878/article/details/136998078>
- <https://blog.csdn.net/powerbond/article/details/132911392>
- <https://blog.csdn.net/soulverzz/article/details/147068430>
- <https://blog.csdn.net/bagell/article/details/145824905>
- https://blog.csdn.net/qq_58718853/article/details/145546524

4.1 环境配置

模型部署的详细环境见 https://github.com/AlenBarry/ZJU_7/requirements.txt。笔者是按照 Qwen2.5-VL 的 github 网站里的 requirements_web_demo.txt 下载的。该 txt 中需要特别注意的几个部分罗列如下：

- **python 版本高于 3.10，torch 版本高于 2.1.** 这一点之所以放在靠前位置，是因为非常重要!!! 由于 gradio 兼容性问题，这两个硬性条件必须满足，否则你当你碰到冲突问题时，之前下载的所有库都需要卸载重装。笔者为此付出了惨痛的代价。
- **git+<https://github.com/huggingface/transformers.git>** 非常容易因为通信问题而卡住导致下不下来，如下图所示

```
(LLM) PS E:\projects\project41-LLM> pip install git+https://github.com/huggingface/transformers.git
Collecting git+https://github.com/huggingface/transformers.git
  Cloning https://github.com/huggingface/transformers.git to c:\users\11515\appdata\local\temp\pip-req-build-3842n7b6
  Running command git clone --filter=blob:none --quiet https://github.com/huggingface/transformers.git 'C:\Users\11515\AppData\Local\Temp\pip-req-build-3842n7b6'
```

图 6: transformers 库下载问题

笔者挂了 vpn 也会出现一样的问题，多次尝试中仅一次成功且无法复现。解决方法是进入 <https://github.com/huggingface/transformers> 网站，下载压缩包到一地址并解压，然后命令行运行

```
$cd .../transformers-main #该文件夹下有文件 setup.py
$pip install .
```

即可。

- **gradio==5.23**. 注意：不能高也不能低，否则运行程序时均会报错：

```
File "D:\AppGallery\Downloads\anaconda\envs\...\lib\site-packages\gradio_client\utils.py", line 911, in json_schema_to_python_type
    type_ = _json_schema_to_python_type(schema, schema.get("$defs"))
File "D:\AppGallery\Downloads\anaconda\envs\...\lib\site-packages\gradio_client\utils.py", line 947, in _json_schema_to_python_type
    [_json_schema_to_python_type(i, defs) for i in items["prefixItems"]]
File "D:\AppGallery\Downloads\anaconda\envs\...\lib\site-packages\gradio_client\utils.py", line 947, in <listcomp>
    [_json_schema_to_python_type(i, defs) for i in items["prefixItems"]]
File "D:\AppGallery\Downloads\anaconda\envs\...\lib\site-packages\gradio_client\utils.py", line 978, in _json_schema_to_python_type
    desc = " | ".join([_json_schema_to_python_type(i, defs) for i in schema["type"]])
File "D:\AppGallery\Downloads\anaconda\envs\...\lib\site-packages\gradio_client\utils.py", line 978, in <listcomp>
    desc = " | ".join([_json_schema_to_python_type(i, defs) for i in schema["type"]])
File "D:\AppGallery\Downloads\anaconda\envs\...\lib\site-packages\gradio_client\utils.py", line 926, in _json_schema_to_python_type
    return _json_schema_to_python_type(defs[schema["$ref"].split("/")[-1]], defs)
File "D:\AppGallery\Downloads\anaconda\envs\...\lib\site-packages\gradio_client\utils.py", line 965, in _json_schema_to_python_type
    des = [
File "D:\AppGallery\Downloads\anaconda\envs\...\lib\site-packages\gradio_client\utils.py", line 966, in <listcomp>
    f"{n}: {_json_schema_to_python_type(v, defs)}{get_desc(v)}"
File "D:\AppGallery\Downloads\anaconda\envs\...\lib\site-packages\gradio_client\utils.py", line 926, in _json_schema_to_python_type
    return _json_schema_to_python_type(defs[schema["$ref"].split("/")[-1]], defs)
File "D:\AppGallery\Downloads\anaconda\envs\...\lib\site-packages\gradio_client\utils.py", line 965, in _json_schema_to_python_type
    des = [
File "D:\AppGallery\Downloads\anaconda\envs\...\lib\site-packages\gradio_client\utils.py", line 966, in <listcomp>
    f"{n}: {_json_schema_to_python_type(v, defs)}{get_desc(v)}"
File "D:\AppGallery\Downloads\anaconda\envs\...\lib\site-packages\gradio_client\utils.py", line 973, in _json_schema_to_python_type
    f"str, {_json_schema_to_python_type(schema['additionalProperties'], defs)}"
File "D:\AppGallery\Downloads\anaconda\envs\...\lib\site-packages\gradio_client\utils.py", line 919, in _json_schema_to_python_type
    type_ = get_type(schema)
File "D:\AppGallery\Downloads\anaconda\envs\...\lib\site-packages\gradio_client\utils.py", line 880, in get_type
    if "const" in schema:
TypeError: argument of type 'bool' is not iterable
```

图 7: gradio 库版本问题

4.2 gradio 部署

gradio 是一个强大的开源 Python 库，用于快速创建和分享机器学习模型的交互式 Web 界面。它允许开发者轻松地将模型部署为可访问的 Web 应用程序，无需具备前端开发知识。Qwen2.5-VL 的 github 仓库内置了相应的函数 `web_demo_mm.py`，配置好环境后运行即可。当然，这是理想中的情况，实际上还需要解决以下问题：

- **文件缺省**。初次运行时系统会报错 ‘Could not create share link. Please check your internet connection or our status page: <https://status.gradio.app>’，而后给出缺省文件名的下载地址、要改的名字和要放置的地址（一般是.../ana -conda/envs/<env_name>/Lib/site-packages/gradio）。下载后改为指定名字（`frpc_linux_amd64_v0.2` 或 `frpc_linux_amd64`

_v0.3, 根据要求选择) 并放置于目标文件夹下即可。注意: 若是 Windows 系统, 一定要去掉.exe 后缀; 若是 Linux 系统, 执行

```
$chmod +x frpc_linux_amd64_v0.3 # or 0.2
```

给予权限。

- **参数调整。**args.share 调为 True, args.checkpoint 可赋值为模型的云端相对路径, 如'Qwen/Qwen2.5-VL-3B-Instruct' 即可。
- **其他。**151 行语句中的 timeout 参数控制模型的最长响应时间, 如果响应超时, 模型会报错。如果硬件设备功能不够强, 可将该参数适当调大。

运行 web_demo_mm.py 后, 访问 <http://127.0.0.1:7860/>, 即可看到 Qwen 模型的可视化界面, 如下图所示。

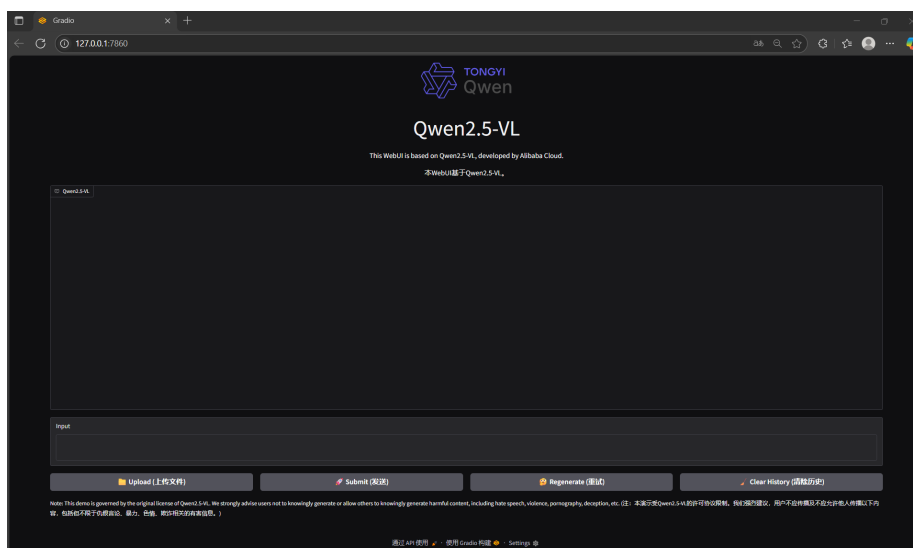


图 8: Qwen2.5-VL 的 gradio 交互界面

模型的对话功能是正常的。然而笔者的本地 cpu 算力实在窘迫, 生成以下一句简单的对话需要超过五分钟

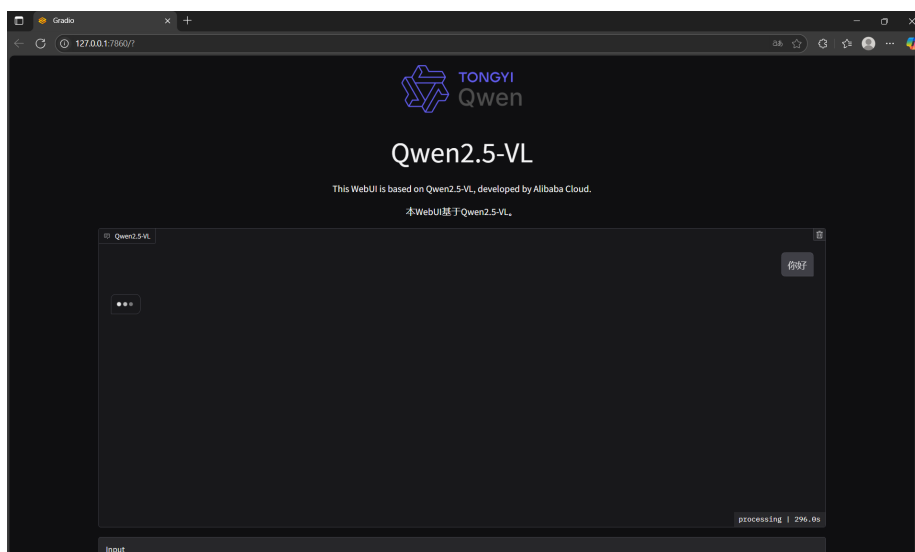


图 9: Qwen2.5-VL 的 gradio 交互示例

这显然不能满足后续任务的要求。所以，笔者换用服务器继续实现本地化部署。由于服务器的端口映射设置较为复杂，笔者只使用下载参数的方式运行。

4.3 参数部署

下载 modelscope 库，并编写下载参数的函数 `download.py` 与模型交互的函数 `run_model.py` 函数。由于正文中贴出所有代码占用空间过大，有碍观瞻，请见附录对应部分。可运行 py 文件请见 https://github.com/AlenBarry/ZJU_7/download.py 与 https://github.com/AlenBarry/ZJU_7/run_model.py。

然而，笔者的服务器的硬件条件仍不满足。在之前的实验环境部分提及，服务器的 CUDA 版本是 11.7，无法兼容 `torch==2.1.0` 及以上版本。迫不得已，笔者只得尝试调用云端部署好的 Qwen2.5-VL-72B 的 API 进行后续任务的实现。

4.4 API 部署

API 的调用则较为简单。在 modelscope 官网注册账号并获取 API-KEY，下载 openai 库并直接调用函数即可，见附录或 https://github.com/AlenBarry/ZJU_7/run_api.py。

模型可以在合理时间范围内正常交互。

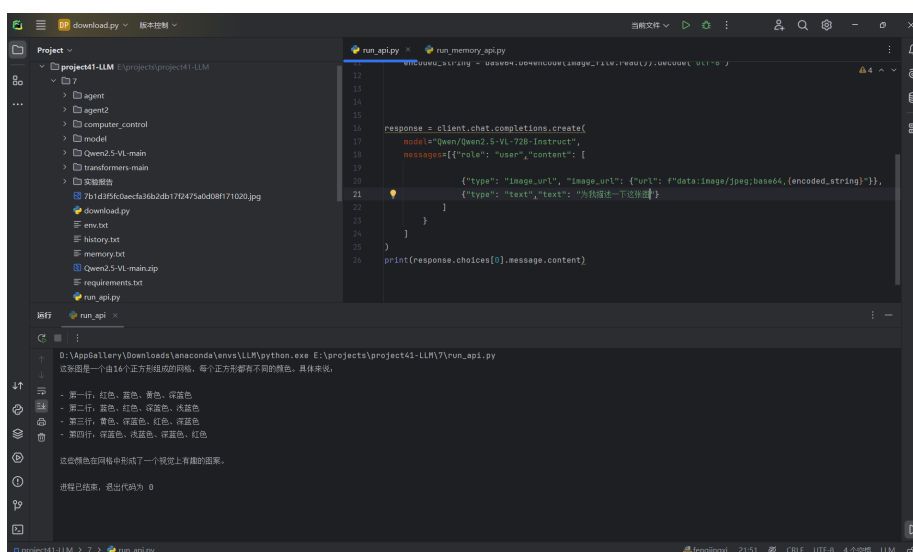


图 10: Qwen2.5-VL 的 API 交互示例

然而，API 端的模型调用是没有记忆功能的。笔者通过在本地记录一个.txt 文件记录对话、并在每一次输入时将整个对话记录一并输入的方式实现了模型的短期记忆能力,具体代码见附录或 https://github.com/AlenBarry/ZJU_7/run_memory_api.py. 附带记忆功能的模型的交互记录如下所示

```

user 你好，我叫常煜杰，很高兴见到你 E:/projects/project41-LLM/7/computer_control/pics/screen.png
assistant 你好，常煜杰！很高兴见到你。从你的电脑桌面来看，你似乎有很多工作和学习相关的软件和文件。看起来你可能对编程、数据分析和图像处理等方面很感兴趣。有什么我可以帮助你的吗？
user 我叫什么名字？ E:/projects/project41-LLM/7/computer_control/pics/screen.png
assistant 你的名字是常煜杰。

```

从 4.3 与 4.4 的代码部分可以看出，本地部署与 API 部署在调用时的区别仅仅在于由输入到输出的实现函数不同，由于笔者设备方面实在贫瘠，后续实验使用 API 调用的方式实现。而 API 调用比之本地化方法的优点则在于：能够选择模型的范围更广；可承受的参数量大幅提升，后续实验中，我们会充分利用这一点，选择性能更加强大的 Qwen2.5-VL-72B 模型作为研究对象。

5 阶段一：图像描述与视觉问答任务

5.1 数据集选择与制作

应题目要求，数据集的组成应为电脑屏幕各个界面的截图。这里我们选择三条需要在阶段三复现成果的指令，并把指令的每一个子步骤的屏幕截图收集起来作为考察数据集。我们称敲击一次键盘、点击一下鼠标为一次操作，指令需满足以下需求：

- 人类操作者只需要通过上述操作的组合就可以实现
- 步骤不得过于繁琐或简单，通常在操作次数 5 到 10 步之内即可
- 具有实用性，生活工作中经常使用

笔者选择的三个指令如下：

- **mission 1:** “给我的 QQ 好友 abc 发送祝福信息”
- **mission 2:** “登录西安交通大学思源学堂, 网址 <https://syxt.xjtu.edu.cn/>, 账号 2204112910, 密码 *****” (出于信息保密, 实际运行时输入为真实密码)
- **mission 3:** “用 Vscode 运行 E:/projects/project41-LLM/7/test.py”

任务一可拆分为 7 步：“双击/QQ 程序” - “左键/登录键” - “左键/abc 好友头像” - “左键/聊天框” - “输入文字/祝福语” - “左键/发送键” - “结束/无约束”

任务二可拆分为 8 步：“双击/firefox 浏览器程序” - “输入文字/目标网址” - “回车/无约束” - “左键/登录窗口” - “输入文字/账号” - “输入文字/密码” - “左键/登录键” - “结束/结束”

任务三可拆分为 9 步：“双击/vscode 程序” - “左键/OpenFolder” - “输入文字/目标地址” - “回车/无约束” - “左键/进入” - “左键/文件夹” - “左键/目标程序” - “左键/运行键” - “结束/结束”

针对三个任务依照以上步骤拆分并逐步截图保存。具体截图及编号见 https://github.com/AlenBarry/ZJU_7/data。

本阶段调用 Qwen2.5-VL-72B 预训练大模型，所有使用的测试数据与程序均可在 github 上找到。测试的目的侧重于寻找模型的能力边界，以更好地指导后续任务的实现。

5.2 图像描述 Captioning

规定输入的语句均为“请描述这张图片”，输入的图片为数据集内所有截图，模型的每次回答不设置记忆功能。交互程序为 https://github.com/AlenBarry/ZJU_7/phase1/captioning.py，与模型的交互记录详见 https://github.com/AlenBarry/ZJU_7/phase1/caption.txt。接下来对模型的输出进行分析

5.2.1 信息识别的准确性

针对当前界面的识别，模型的准确率达到到了 100%。模型能够准确的识别出数据集中每一张图片的几乎所有内容，且精度十分高，能够准度的读取图片中的文字并输出其对应的功能。同时，模型的幻觉现象并没有出现，模型并不会自主生成图片中不存在的信息和要素，图片中各个部分的相对位置的识别也较为准确。

5.2.2 能力边界：“注意力”涣散

数据集中，并不是所有的应用界面打开都是全屏显示的。以打开 Vscode 编译器的图片 data/mission3/step2.png 为例，如下所示

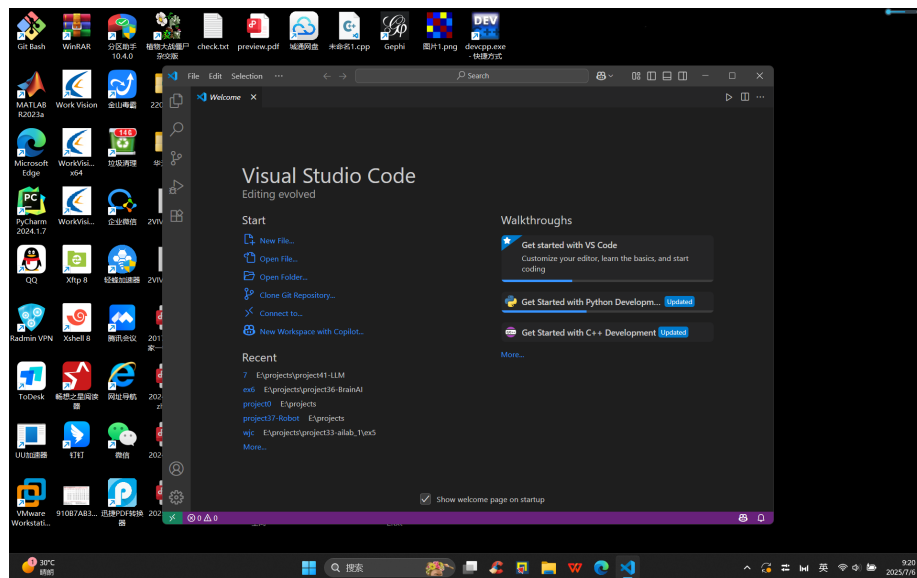


图 11: .../data/mission3/step2.png

Input Sentence: 请描述这张图片

Input Picture: E:/projects/project41-LLM/7/data\mission3\step2.png

Output: 这张图片显示了一个电脑桌面，桌面上有许多应用程序和文件的图标。在桌面中央，有一个打开的 Visual Studio Code (VS Code) 窗口。

桌面部分：

- 左侧有多个应用程序图标，包括 Git Bash、WinRAR、MATLAB R2023a、Microsoft Edge、PyCharm 2024.1.7、QQ、ToDesk、钉钉、微信等。
- 右上角有一些文件和快捷方式图标，如 check.txt、preview.pdf、未命名1.cpp、Gephi、图片1.png、devcpp.exe 等。

Visual Studio Code 窗口：

- 窗口标题为 “Welcome”，显示了 VS Code 的欢迎页面。
- 页面左侧有 “Start” 部分，包含以下选项：
 - New File
 - Open File
 - Open Folder
 - Clone Git Repository
 - Connect to...
 - New Workspace with Copilot
- 页面右侧有 “Walkthroughs” 部分，包含以下选项：
 - Get started with VS Code
 - Get Started with Python Development (Updated)
 - Get Started with C++ Development (Updated)
- 页面底部有 “Recent” 部分，列出了最近打开的项目和文件路径。
- 底部状态栏显示了一些信息，如当前温度 (30°C)、日期 (2025年7月6日) 等。

整体来看，这张图片展示了一个编程开发环境，用户正在使用 VS Code 进行代码编辑和开发工作。

可以看出，尽管编译器的界面占据了屏幕的绝大部分，模型仍然列举出背景桌面上较多的应用名称，显得不分主次。在执行实际指令过程中，有相当多的应用界面是不占据全部屏幕的，其中更有显示界面比 Vscode 编译器还要小的窗口，如 QQ 的登录窗口 (mission1/step2.png)，如下所示

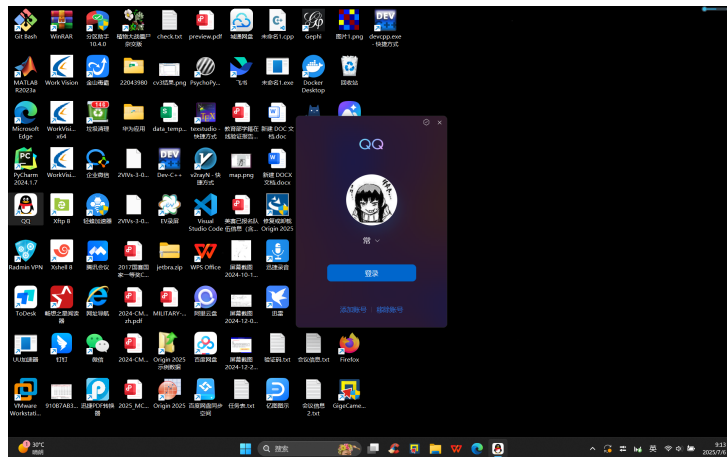


图 12: .../data/mission1/step2.png

Input Sentence: 请描述这张图片

Input Picture: E:/projects/project41-LLM/7/data\mission1\step2.png

Output: 这张图片显示了一个电脑桌面，桌面上有许多文件和应用程序的图标。背景是黑色的，图标排列得比较密集。在屏幕的右侧中央位置，有一个QQ登录窗口，窗口中显示了一个动漫风格的头像，并且有“登录”按钮以及“添加账号”和“移除账号”的选项。

桌面上的图标包括各种软件和文件夹，例如Git Bash、WinRAR、分区助手、植物大战僵尸杂交版、Gephi、Docker Desktop、MATLAB R2023a、Work Vision、金山毒霸、Microsoft Edge、PyCharm 2024.1.7、企业微信、Visual Studio Code、Radmin VPN、Xshell 8、ToDesk、钉钉、微信、Firefox浏览器等。还有一些文档和压缩包文件，如check.txt、preview.pdf、未命名1.cpp、cv3结果.png、data_temp.xlsx、新建DOC文档.docx、2VIVs-3-0...、jebtra.zip、2024-CM...zh.pdf、2024-CM...Origin 2025示例数据、910B7AB3... 迅速PDF转换器、2025_MC... Origin 2025 百度网盘同步空间等。

此外，桌面上还有百度网盘、阿里云盘、WPS Office、EV录屏、轻蜂加速器、迅雷等常用工具的快捷方式。任务栏位于屏幕底部，显示了当前的时间为2025年7月6日9点13分，天气晴朗，温度为30℃。

可以看出，模型已经将描述的重心放在了背景上，并不能够聚焦于当前桌面最顶部的应用窗口。这在执行涉及定位和生成式任务的指令时是需要加以避免的，无关的场外信息会造成严重的干扰。

5.2.3 能力边界：相似图片区分

数据集中有两张特殊的图片 data/mission1/step4.png 和 data/mission1/step5.png，它们极为相似却表示完全不同的含义，如下图所示

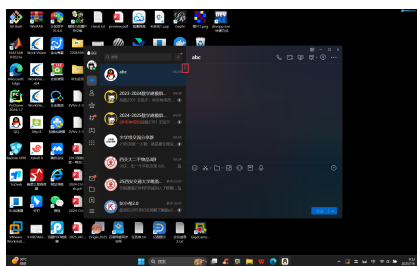


图 13: mission1/step4.png

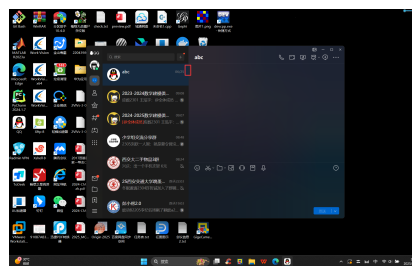


图 14: mission1/step5.png

Input Sentence: 请描述这张图片

Input Picture: E:/projects/project41-LLM/7/data\mission1\step4.png

Output: 这张图片显示了一个电脑桌面，桌面上有许多应用程序和文件的图标。这些图标包括各种软件如Git Bash、WinRAR、MATLAB、Microsoft Edge、PyCharm等，以及一些文档和文件夹。在桌面的右侧，有一个打开的即时通讯软件窗口，显示了多个聊天群组 and 联系人列表。当前选中的聊天对象是名为“abc”的用户，但聊天内容为空。此外，任务栏上也显示了一些常用的应用程序图标和系统信息，如天气、时间和网络状态等。整体来看，这似乎是一个用于编程和学习的工作环境。

Input Sentence: 请描述这张图片

Input Picture: E:/projects/project41-LLM/7/data\mission1\step5.png

Output: 这张图片显示了一个电脑桌面，桌面上有许多应用程序和文件的图标。这些图标包括各种开发工具、办公软件、浏览器、文件夹等。在桌面中央，有一个打开的即时通讯软件窗口，显示的是一个名为“abc”的聊天界面。聊天列表中列出了多个群聊和联系人，其中一些群聊名称与数学建模相关，如“2023-2024 数学建模”和“2024-2025 数学建模”。此外，还有一些其他群聊和联系人，如“小学组交流分享群”、“西交大二手物品3群”等。聊天窗口的右侧是聊天内容区域，但当前没有显示任何消息。整个桌面背景为黑色，图标和窗口以深色为主，符合暗色主题的设置。

step4.png 表示登入 QQ 后在左侧联系人点击目标好友的状态，此时直接可操作界面位于 QQ 左侧的联系人窗口，而 step5.png 表示左键点击聊天框准备进行内容的输入。然而模型无法捕捉到图片上细小的不同点，对它们的描述基本相同，这很可能导致连贯操作中指令的混淆或者重复。

5.2.4 可能的解决措施

- 使用更详尽的提示词：“输入图片是一张电脑屏幕截图，请详细描述电脑最上层的应用”；
- 启用历史对话功能，要求模型进行不重复的分析，如“观察输入图片有哪些变化，猜测电脑操作者的目的”。

这些提示词和方法将在 VQA 任务中进行进一步的测试。

5.3 视觉问答 (Visual Question Answering)

规定输入的语句分别为“输入图片是一张电脑屏幕截图，请详细描述电脑最上层的应用”和“观察输入图片有哪些变化，猜测电脑操作者的目的”，输入的图片为数据集内所有截图，每个任务中模型的每次回答设置记忆功能。交互程序为 https://github.com/AlenBarry/ZJU_7/phase1/vqa.py，与模型的交互记录详见 https://github.com/AlenBarry/ZJU_7/phase1/vqa1.txt 与 https://github.com/AlenBarry/ZJU_7/phase1/vqa2.txt。接下来对模型的输出进行分析

5.3.1 顶层应用解析

在提示词中加入“顶层应用”相关语句后，模型将更多注意力集中在应用软件的界面上，忽略了对屏幕背景的观察。对于 mission1/step1.png, mission2/step1.png 和 mission3/step1.png 三个初始状态，模型将“顶层”的语义识别为桌面上最上、最靠左的应用图标 Git Bash。

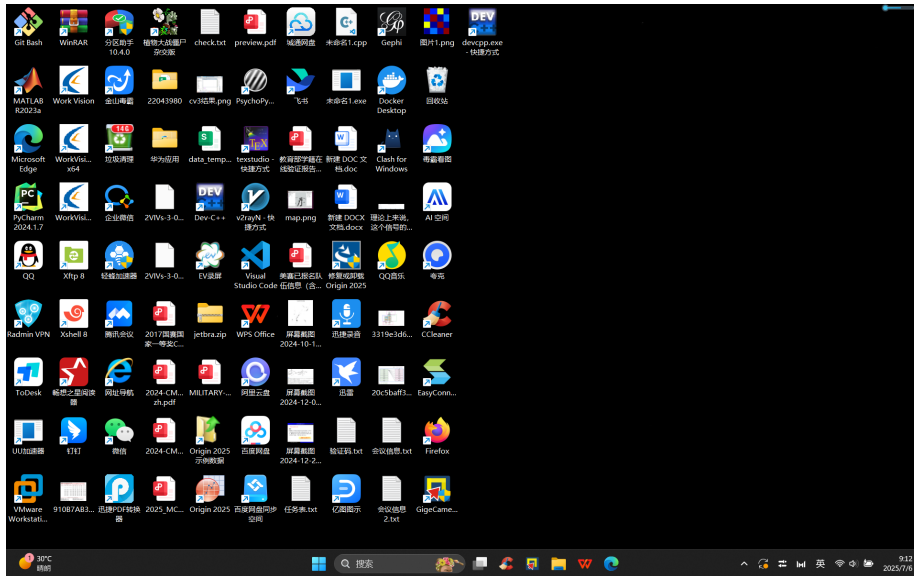


图 15: data/mission1&mission2&mission3/step1.png

Input Sentence: 输入图片是一张电脑屏幕截图，请详细描述电脑最上层的应用

Input Picture: E:/projects/project41-LLM/7/data\mission1\step1.png

Output: 从图片中可以看出，电脑最上层的应用是“Git Bash”。Git Bash 是一个用于在 Windows 上运行 Git 命令行工具的程序，它提供了一个类 Unix 的命令行界面，使得用户可以在 Windows 系统上使用 Git 进行版本控制操作。它的图标通常是一个绿色的背景上有一个白色的“bash”字样。在这个截图中，Git Bash 的图标位于屏幕左上角的第一行第一个位置。

#其余两个mission的step1.png输出完全相同

除初始状态外的其余状态均准确识别。总体来说，模型描述应用界面的准确程度是可以接受的。

5.3.2 能力边界：细粒度图片理解

对于图片 data/mission1/step4.png 和 data/mission1/step5.png，在两个提示词的辅助下，模型依然无法辨别出它们二者的区别：

Input Sentence: 观察输入图片有哪些变化，猜测电脑操作者的目的

Input Picture: E:/projects/project41-LLM/7/data\mission1\step4.png

Output: 从图片中可以看到，QQ 软件已经成功登录，并且显示了多个聊天窗口和群组。与之前的图片相比，主要变化是 QQ 界面的展示，这表明电脑操作者可能有以下目的：

1. **查看消息**：操作者可能需要查看和回复未读的消息，例如“abc”、“2023-2024 数学建模...”等聊天窗口中的新消息。
2. **参与讨论**：操作者可能需要参与到某个群组的讨论中，如“2023-2024 数学建

模...”或“2024-2025 数学建模...”，这些群组可能与学术研究、项目合作或学习交流有关。

3. ****文件传输****: 在QQ中，操作者可能需要发送或接收文件，例如文档、代码或其他相关资料，以支持他们的工作或学习。

4. ****协调安排****: 通过QQ群组，操作者可能需要与其他成员协调活动安排、会议时间或任务分工等事宜。

5. ****获取信息****: 操作者可能需要从QQ群组中获取最新的通知、公告或资源分享，以便及时了解相关信息。

6. ****社交互动****: 除了工作和学习相关的交流，操作者也可能通过QQ进行日常的社交互动，与朋友或同事保持联系。

综上所述，QQ界面的展示表明操作者正在使用QQ进行多方面的沟通和协作，具体目的可能包括查看消息、参与讨论、文件传输、协调安排、获取信息以及社交互动等。

Input Sentence: 观察输入图片有哪些变化，猜测电脑操作者的目的

Input Picture: E:/projects/project41-LLM/7/data/mission1/step5.png

Output: 从图片中可以看到，QQ软件已经成功登录，并且显示了多个聊天窗口和群组。与之前的图片相比，主要变化是QQ界面的展示，这表明电脑操作者可能有以下目的：

1. ****查看消息****: 操作者可能需要查看和回复未读的消息，例如“abc”、“2023-2024 数学建模...”等聊天窗口中的新消息。

2. ****参与讨论****: 操作者可能需要参与到某个群组的讨论中，如“2023-2024 数学建模...”或“2024-2025 数学建模...”，这些群组可能与学术研究、项目合作或学习交流有关。

3. ****文件传输****: 在QQ中，操作者可能需要发送或接收文件，例如文档、代码或其他相关资料，以支持他们的工作或学习。

4. ****协调安排****: 通过QQ群组，操作者可能需要与其他成员协调活动安排、会议时间或任务分工等事宜。

5. ****获取信息****: 操作者可能需要从QQ群组中获取最新的通知、公告或资源分享，以便及时了解相关信息。

6. ****社交互动****: 除了工作和学习相关的交流，操作者也可能通过QQ进行日常的社交互动，与朋友或同事保持联系。

综上所述，QQ界面的展示表明操作者正在使用QQ进行多方面的沟通和协作，具体目的可能包括查看消息、参与讨论、文件传输、协调安排、获取信息以及社交互动等。

模型没有发现操作区域的细微变化，输出依然一模一样。这表明模型难以进行细粒度的图像区分与理解。

5.3.3 能力边界：上下文关联

关联图片的上下文对话几乎是模型的重灾区。模型无法准确捕捉到图片的变化从而正确的揣测电脑操作者的意图。图片 data/mission1/step2.png 到 data/mission1/step7.png 剧烈变化，但输出却几乎一模一样。

这表明上下文（尤其是含有图片且输出较长文字的对话）会对模型的连续生成造成负面干扰。模型对历史信息的依赖程度过于严重，容易出现输出内容重复的恶劣情况。

5.3.4 启示

模型在细粒度图像识别和上下文关联上的贫瘠无疑是我们将要解决的重点问题。我们需要对这些模型本身固有的缺陷加以利用，来完成阶段三更加复杂的任务。对此，有如下启示：

- GUI 任务不可能由一个大模型单独完成。需要将它至少拆分成“定位-操作”两个子任务，每个子任务由至少一个大模型完成，它们之间并不共享记忆。需要有程序在中间做输入和输出的处理；
- 更进一步，由于模型无法胜任细粒度图像识别，“定位”任务几乎不可能由一个大模型单独完成，考虑使用二重定位的方式，由一个大模型筛选出粗略的区间，另一个大模型再在该区间中进一步筛选。这一想法会在第六部分进行尝试；
- 操作者的最终目的必须以 system 或 user 提示词的方式植入每一个大模型。仅靠上下文关联，模型无法自发地预测、判断目的达成与否，必须要有信息的加强。

6 阶段二：自然语言指令到 UI 元素坐标的转换

6.1 屏幕操作

之前我们在预设要实现的功能中，有这样一条准则：“人类操作者只需要通过敲击键盘、点击鼠标操作的组合就可以实现”。我们需要做的是：通过编程实现这些操作，以及将这些操作整合成模型能理解的格式。

6.1.1 操作指令的 python 实现

使用 python 的两个库 pyautogui 和 pyperclip 可以完成要求。

pyautogui 库是一个跨平台的 GUI 自动化工具，它允许用户通过编程的方式控制鼠标和键盘，从而实现自动化操作。使用 pyautogui，你可以编

写脚本来模拟点击、拖拽、输入文本等动作。此外，pyautogui 还提供了屏幕截图和图像识别功能，可以帮助实现在屏幕上查找特定的图像并进行相应的操作。

而 pyperclip 库则专注于处理剪贴板内容。它可以将文本复制到剪贴板，或者从剪贴板粘贴文本到你的程序中。这对于需要频繁处理文本数据的应用程序来说特别方便，比如在自动化数据提取或转换过程中，pyperclip 可以帮助你快速获取和设置剪贴板中的内容。

基于这两个库的操作函数请见 https://github.com/AlenBarry/ZJU_7/phase2_command/computer_control.py

6.1.2 操作指令的可理解格式

笔者尝试将多种指令组合进行拆分和总结，并作为 system 提示词输入给大模型，如下：

现假设你是一位电脑操作员，输入图片是你的电脑屏幕截图，输入语句是你想要实现的操作，请在下列指令集中选择你立刻要执行的指令

Double Click: 表示鼠标左键双击

Left Click: 表示鼠标左键单击

Right Click: 表示鼠标右键单击

Type: 表示输入文字。对于输入的内容，请用 ; 符号与指令分隔开

Enter: 表示点击回车键

END: 表示目标已达成，无需任何操作

要求只输出一个指令，输出不含其他任何词句

具体代码见 phase2_command/command.py, 模型的交互结果见 phase2_command/result_v1。可以看出，模型能够依据指令来输出对应的格式。以上提示词看似简洁严谨，但经简单统计，正确率只有可怜的 70.83% (17/24)。对出错类型进行归类，可以分成以下三类问题：

- 模型无法区分何时单击，何时双击。模型在三个任务的第一步都选择了左键，这样无法打开指定的软件。例如

Input Sentence: 给我的QQ好友abc发送祝福信息

Input Picture: E:/projects/project41-LLM/7/data/mission1/step1.png

Output: Left Click

- 模型图像识别的粒度太粗，它无法从图片中认知到一个事实：在执行 Type 操作时需要先选中文字框，中间的左键衔接步骤总会被错误的判断为 Type。例如之前的图 13 (mission1/step4.png)，在当前步骤应当选择先左击聊天框进行切换，但模型选择了敲字

Input Sentence: 给我的QQ好友abc发送祝福信息

Input Picture: E:/projects/project41-LLM/7/data/mission1/step4.png

Output: Type: 祝福信息;

- “多米诺”式错误。由于第二种错误提前将一些信息 Type 了出去，在应当 Type 的步骤，上下文信息会阻止模型的 Type 选择，从而造成更多的错误。例如之前的图 14 (mission1/step5.png)，在当前步骤应当选择输入生成文字，但模型通过上下文判断码字已经结束，反而选择了左键

Input Sentence: 给我的QQ好友abc发送祝福信息
 Input Picture: E:/projects/project41-LLM/7/data\mission1\step5.png
 Output: Left Click

可能的解决措施是依旧从 system 提示词出发，将模型缺失的两种信息作为先验信息加以提示：

注意：从桌面打开应用时需要双击；
 Type 指令执行前，需要先 Left Click 输入文本位置的文字框

加入以上段落提示此后，生成指令的准确率提高到了 91.67% (22/24)。结果详见 phase2_command/result_v2

然而，问题依然严重：模型几乎不选择 Type 指令了，而其他指令的生成格式出现了问题：

Input Sentence: 登录西安交通大学思源学堂，网址<https://syxt.xjtu.edu.cn/>，账号2204112910，密码*****
 Input Picture: E:/projects/project41-LLM/7/data\mission2\step1.png
 Output: Double Click;Microsoft Edge

Input Sentence: 登录西安交通大学思源学堂，网址<https://syxt.xjtu.edu.cn/>，账号2204112910，密码*****
 Input Picture: E:/projects/project41-LLM/7/data\mission2\step4.png
 Output: Left Click;登录入口》

Input Sentence: 登录西安交通大学思源学堂，网址<https://syxt.xjtu.edu.cn/>，账号2204112910，密码*****
 Input Picture: E:/projects/project41-LLM/7/data\mission2\step5.png
 Output: Left Click;学工号/手机号/NetID/身份证(Staff/Student ID/I

这表示模型在对 Type 和其他操作游移不定。

分辨左键和 Type 超出了模型的能力，笔者采用了更加底层的修复办法：将提示词的最后半句“Type 指令执行前，需要先 Left Click 输入文本位置的文字框”替换为“除了 Type 之外的指令，输出后不要附加任何文字”，并将 Type 指令的描述改为：

Type: 表示左键后输入文字。对于输入的内容，请用；符号与指令分隔开

第二次修正后的结果见 phase2_command/result_v3。这次修正后，指令的正确率达到了 100% (24/24)，完成了这一阶段的任务。

6.2 屏幕的 UI 定位

6.2.1 电脑屏幕 UI

电脑 UI 中的坐标系统用于确定界面上各个元素的位置，通常采用二维坐标系，以屏幕的左上角为原点 (0, 0)，X 轴向右递增，Y 轴向下递增。通过指定元素在 X 轴和 Y 轴上的坐标值，可以精确地定位按钮、文本框、图像等界面组件的位置，确保布局合理且用户操作便捷。

在电脑 UI 的坐标系统中，X 和 Y 坐标的取值范围取决于屏幕的分辨率。例如，如果屏幕分辨率为 1920x1080 像素，那么 X 坐标的取值范围是 0 到 1919，Y 坐标的取值范围是 0 到 1079。每个坐标值对应屏幕上的一个像素点，通过这些坐标值可以精确地定位和布局界面元素。不同的显示器和窗口大小会有不同的分辨率，因此坐标范围也会相应变化。笔者使用的电脑分辨率是 1920x1200 的。

之前在 6.1 种介绍过的 pyautogui 库的各种功能的作用区域就是由电脑屏幕的 UI 坐标确定的。这个库还提供全屏截屏和指定 UI 区域截屏两个功能，方便我们进行接下来的操作。

6.2.2 屏幕 UI 的模型定位

设置 system 提示词如下：

现假设你是一位电脑操作员，输入图片是你的电脑屏幕截图，输入语句是你想要实现的操作。请根据图片选择你接下来要用鼠标操作的区域，并输出这块区域的 UI 坐标。输出的格式形如：[x,y,w,h]，它们表示该区域左上角和右下角的 UI 坐标。要求只输出一个指令，输出不含其他任何词句。区域可以大一些，但它的中心一定要尽可能靠近你想要用鼠标操作的地点。

交互程序为 phase2_UI/UI_location.py, 运行结果见 phase2_UI/result_v1. 大模型的区域识别准确率为 70.83%(17/24)。

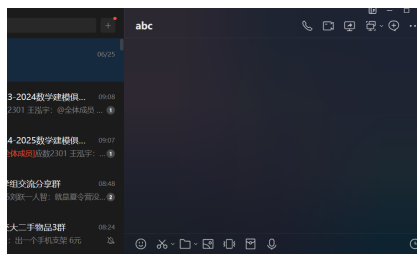


图 16: result_v1/1_step4.png

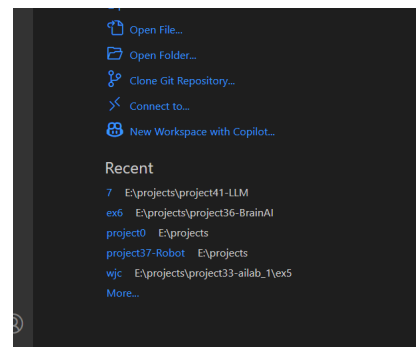


图 17: result_v1/3_step2.png

从以上谬误图例可以看出，可行区域的确在输出的 UI 区间之中，但它们由于自身的占地区域太小而无法处于操作的中心位置。

这时，先前第五部分实验得到的结论给予了笔者启发：使用两个大模型进行二次区间筛选。第二个大模型的 system 提示词设置如下：

现假设你是一位电脑操作员，输入图片是经过裁剪后的一部分屏幕截图，图片中已经包含了你需要操作的区域。你的任务是进一步确定该区域中鼠标应该点击的位置，并提供一个更加精确的 UI 坐标。请在给定的裁剪区域内，找出你认为最精确的操作区域，并输出该区域的 UI 坐标。"坐标应包括该区域左上角的像素位置以及区域的宽高。输出的格式应为：`[x,y,w,h]`。要求：输出的区域应该尽可能精确，且与操作意图的中心对齐。不要输出其他词句。

然而，出乎意料的是，识别的准确率并没有变得更好，反而变得更加差了，几乎没有区域的中心是对准在操作位置上的，具体的结果请见 phase2_UI/result_v2。

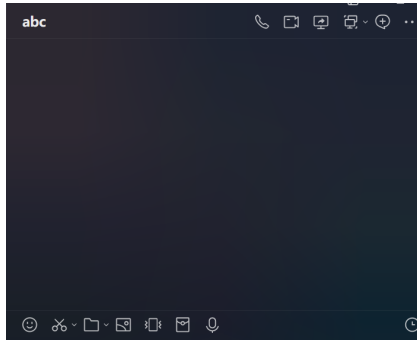


图 18: result_v2/1_step4.png

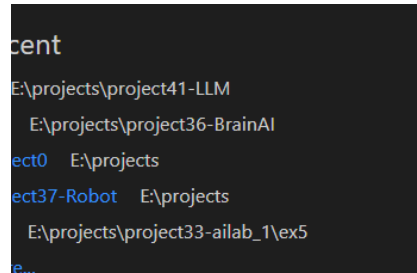


图 19: result_v2/3_step2.png

推测可能的原因是在经过第一个模型的裁剪之后，模型难以理解这张局部截图所传达的信息。必须要有一个精细到操作的输入信息作为指引，才能够使得模型的定位变得精准。

因此，我们在系统中引入第三个大模型，它的作用和提示词与 6.1.2 完全相同。同时，将二次筛选大模型的输入改为“在电脑屏幕 UI 区间 region1 执行操作 output3”，其中 region1 第一个大模型输出到粗糙区间表示，而 output3 为新引入大模型的输出。输出结果见 phase2_UI/result_v3。

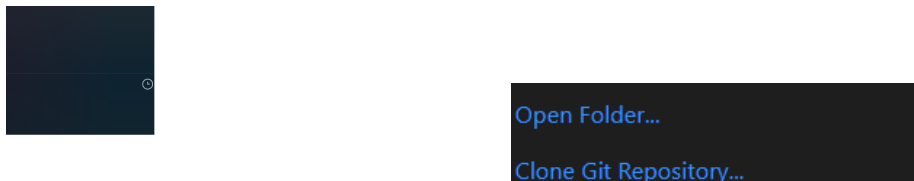


图 21: result_v3/3_step2.png

图 20: result_v3/1_step4.png

可以看出，结果变得更加随机与参差不齐。mission3/step2.png 的结果有所好转，但 mission1/step4.png 的结果变得更加差，截图的范围甚至超出了粗略区间。整体准确率仍远远低于修改前的方案一。UI 区间准确定位这一问题仍然需要更进一步的详细研究。

7 阶段三：桌面自动化流程模拟

7.1 操作-定位 (Operation-Location)GUI 智能体

将 6.1.2 中的方案三和 6.2.2 中的方案一加以组装，即可建立其一个具有基本功能的智能体。该智能体的工作流如图 22 所示。

详细的构建代码请见 phase3/agent1 文件夹，运行 agent_engine.py 即可。在 7.1 智能体的大量测试过程中，笔者发现了以下现象

- 由第六部分的对比实验可以看出，选择操作算子任务对一个大模型来说是完全可以胜任的，几乎不会出错；而细粒度的定位任务则是由于大模型本身的局限导致的，单独使用一个大模型并不能使得定位性能产生显著的改善，而使用两个大模型获取操作和定位带来的时间延迟过于的长；
- 智能体无法正确评估当前所处的状态，如果因定位的不准确而做出无意义的操作，智能体并不会开始尝试其他的方法，而是会一直重复在当前错误的操作之中。这种无意义的死循环是必须要避免的。

因此，基于以上因素，笔者对当前智能体的结构进行了优化。

GUI智能体 workflow

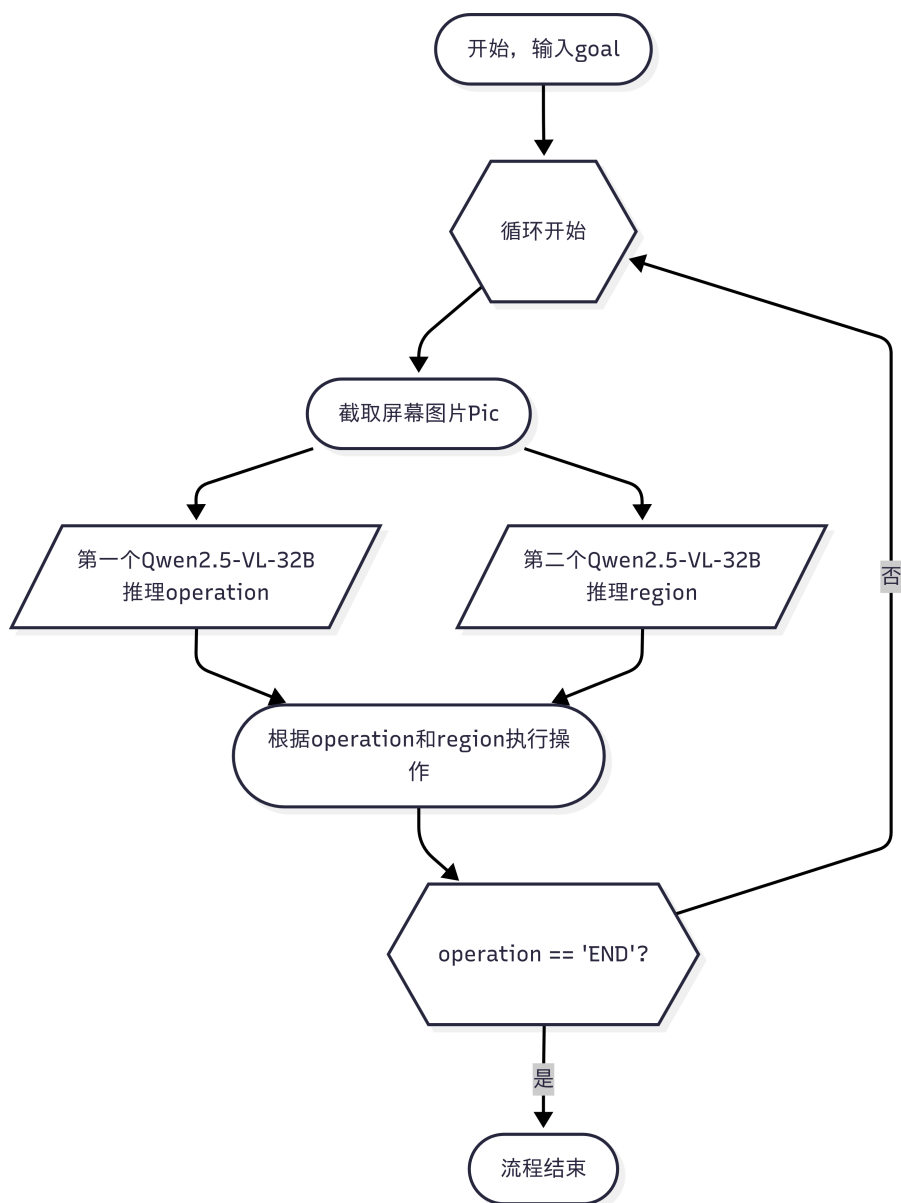


图 22: O-L Agent WorkFlow

7.2 动作识别-行动-预期 (Thought-Action-Expectation)GUI 智能体

该智能体是基于 7.1 的改进版本。

第一个大模型负责根据屏幕截图和第三个大模型的输出，判断上一步操作是否达到预期，然后根据判断输出一个立即需要执行的“想法” thought 给第二个大模型；

第二个大模型是 7.1 中两个大模型的结合体，同时输出操作指令以及该指令作用的位置，统称为 action；

第三个大模型通过第一个大模型输出的动作和第二个大模型输出的操作来对当前操作进行预期描述 expectation，并传递给第一个大模型进行下一个循环。

智能体的工作流如图 23 所示。

详细的构建代码请见 phase3/agent2 文件夹，运行 agent_engine.py 即可。

7.3 智能体任务测试与对比

在这两个智能体上实验我们一开始预设好的三个 mission，每个 mission 执行 5 次，其结果如下表所示

表 1: 智能体任务性能测试表

Accuracy	MISSION1	MISSION2	MISSION3
OL Agent	20%	100%	0%
TAE Agent	60%	80%	0%

OL 智能体在执行 mission1 时，总是会在“单击聊天框输入文本”和“点击发送键”这两个步骤卡住，而后一直循环在“Left Click”和“Type”两个操作，直到交互次数达到上限。

对于 mission2，它可以较完美地执行成功。

对于 mission3，OL 智能体的能力较差，有四次它定位不到 Vscode 的 Open Folder 键，一直循环“Left Click”点击无效位置直到结束；有一次它误打误撞地打开了 Open File 键，并成功输入了文件所在地址，但它将下一步应当执行的“Enter”指令判断为“Left Click”打开键，并陷入了“Left Click”和“Type”之间的循环，直到次数到达上限。

GUI智能体任务 workflow

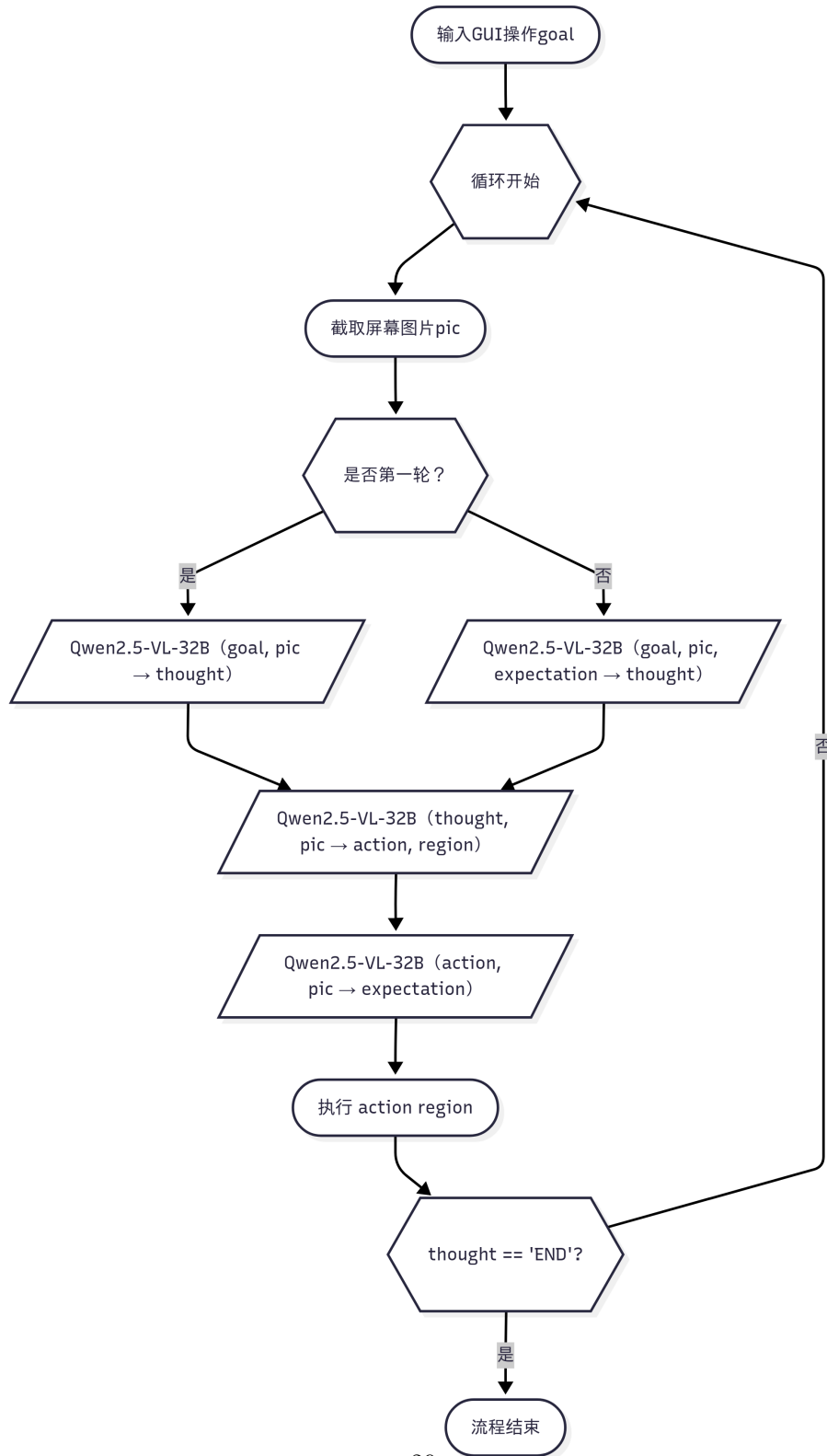


图 23: T-A-E Agent Workflow

TAE 智能体能够比 OL 智能体更好地胜任 mission1，但仍然会在较少的时候无法点击到发送键失误。它在失误后不会像之前的 OL 智能体一样陷入动作的死循环，反而会更加侧重上文信息，提前发出 END 信号。

对于 mission2，它可以较完美地执行成功。

对于 mission3，多数情况下 TAE 智能体能够定位好 Open Folder 键和打开键，但依旧无法较好地定位到左侧的 test.py 和右上角较小的运行符号，总是在打开其他文件或文件夹后唐突结束。

值得注意的是，大模型的预期让智能体在选择区域时更具有导向性，略微提升了准确度，但提升幅度过于有限。TAE 智能体一个较好的优点在于，操作出错后，智能体越来越容易提前给出“END”而不是在错误操作中循环，以避免对本机数据造成不可逆的伤害。

8 总结

本实验围绕利用轻量级视觉语言模型（VLM）完成桌面自动化任务展开，使用 Qwen2.5 - VL - 3B 模型分三个阶段进行探索。首先在阶段一，完成模型本地部署与加载，通过输入桌面应用截图，测试并分析模型在图像描述（Captioning）和视觉问答（VQA）任务的表现，发现其存在“注意力”涣散、相似图片区分困难等能力边界，并提出使用更详尽提示词等解决措施。接着阶段二，实现屏幕操作的 Python 编程与操作指令格式转换，将自然语言指令转化为模型可理解格式并解析 UI 元素坐标，通过优化提示词提升指令生成准确率，还尝试使用多个大模型进行 UI 坐标二次筛选定位。最后阶段三，构建“操作 - 定位（O - L）”和“动作识别 - 行动 - 预期（T - A - E）”两种 GUI 智能体，模拟桌面自动化流程，测试显示 TAE 智能体在任务执行中表现更优，虽大模型预期提升了一定准确度，但提升幅度有限。实验过程中还完成了环境配置与模型部署，包括本地和服务器环境搭建、gradio 部署、参数下载及 API 调用等，并针对部署中遇到的问题如依赖库版本冲突等进行解决。

最后，笔者还有一些自己的心里话想说。在笔者时间第三阶段的过程中，构建的智能体远远不止本文档中记录的这两种，其中更有性能超越其上的智能体，但笔者将这些部分舍去了。在实验的中后期，为了尽可能的实现笔者假设的三个 mission，笔者不断地在智能体中增添大模型，提示词也变得又臭又长。当第一个做出了 mission3 的智能体完成时，笔者并没有感到欣慰，只感到了解脱，对机械性的、重复性的提示词工程的解脱。可笑的是，仅仅更换 mission1 任务中好友的名字，这个看似“强大”的智能体瞬间又变的一塌糊涂。笔者认为这样罔论复杂度、解释性差、依靠机械性工作堆垒来换取性能提升的工作是毫无意义的，同时也是毫无启发性的。这个项目并不是一个面向产品的过程，而是一个面向探索的过程，我们能够从中更深入的理解大模型认知信息的方式才是其最根本的目的。这也算是笔者对将来能够接触到的研究和学习的一些小小寄托和期许吧。这篇报告的摘要部分、总结部分的第一段和两张智能体 workflow 是由大模型生成的，其余部分完全是笔者自己码的字，还望老师多多包涵。如果您能够有耐心看到这里，笔者将感到无上的荣幸。这也是笔者第一次尝试将自己做的项目上传到 github 上，如果您觉得不错，请打个星吧。